

3 - 数据的运算及运算部件

作者：Klein White

时间：2025-04-21

概要：

补码的加减法

◆补码加法： $[X+Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$

◆补码减法： $[X-Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$

◆补码运算的基本规则

- 参加运算的各个操作数均以补码表示，运算结果仍以补码表示
- 符号位和数值位一样参加运算
- 若求和，将两补码直接相加，得两数之和的补码；若求差，则将减数变补(由 $[Y]_{\text{补}}$ 求 $[-Y]_{\text{补}}$)，然后与被减数相加，得两数之差的补码

其中 $-Y$ 的补码等于 Y 的补码连同符号位在内取反加1

加减法的溢出检测：

◎方法1：设 $[X]_{\text{补}}=X_0, X_1, X_2 \dots X_n$, $[Y]_{\text{补}}=Y_0, Y_1, Y_2 \dots Y_n$
其和（或差）为 $[S]_{\text{补}}=S_0, S_1, S_2 \dots S_n$ 。则溢出条件为：

$$V = X_0 Y_0 \bar{S}_0 + \bar{X}_0 \bar{Y}_0 S_0$$

◎方法2：采用变形补码（双符号位补码）

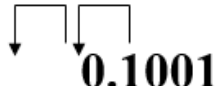
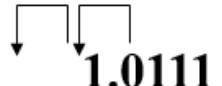
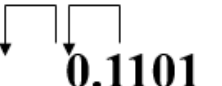
00.1001	11.0111	00.1101	11.0011
+ 00.0011	+ 11.1101	+ 00.1001	+ 11.0111
00.1100	11.0100	01.0110	10.1010

可见，当结果两位符号的值不一致时表时溢出，若设两符号位 S_{f1} 和 S_{f2} ，则

$$V = S_{f1} \oplus S_{f2}$$

◎方法3：采用双高位判断，设：

数值部分最高位产生的进位为 C_s ；符号部分向上产生的进位为 C_f ；仍以上面四个标式为例。

$C_f=0 \ C_s=0$	$C_f=1 \ C_s=1$	$C_f=0 \ C_s=1$	$C_f=1 \ C_s=0$
			
0.1001	1.0111	0.1101	1.0011
+ 0.0011	+ 1.1101	+ 0.1001	+ 1.0111
0.1100	1.0100	1.0110	0.1010

溢出条件为公式： $V = C_f \oplus C_s$

方法三也就是看数值的高位和符号位的进位情况是否一致

浮点数运算

IEEE754标准规定的五种异常情况

① 无效运算 (无意义)

- 运算时有一个数是非有限数, 如:
加 / 减 ∞ 、 $0 \times \infty$ 、 ∞/∞ 等
- 结果无效, 如:
源操作数是NaN、 $0/0$ 、 $x \text{ REM } 0$ 、 $\infty \text{ REM } y$ 等

② 除以0 (即: 无穷大)

③ 数太大 (阶码上溢): 对于SP, 阶码 $E > 1111\ 1110$ (阶大于127)

④ 数太小 (阶码下溢): 对于SP, 阶码 $E < 0000\ 0001$ (阶小于-126-23)

⑤ 结果不精确 (舍入时引起), 例如 $1/3$, $1/10$ 等不能精确表示成浮点数

上述情况硬件可以捕捉到, 因此这些异常可设定让硬件处理, 也可设定让软件处理。让硬件处理时, 称为硬件陷阱。

硬件陷阱: 事先设定好是否要进行硬件处理 (即挖一个陷阱, 设置自陷允许标志), 当执行指令过程中检测到出现相应异常时, 就由硬件自动进行相应的异常响应 (掉入陷阱)。

2025/7/25

浮点数加减法

对阶操作: 让两个数的阶码相同, 阶码小的向阶码大的看齐

$$\begin{aligned} 1.123 \times 10^5 + 2.560 \times 10^2 &= 1.123 \times 10^5 + 0.002560 \times 10^5 \\ &= (1.123 + 0.00256) \times 10^5 = 1.12556 \times 10^5 \\ &= 1.126 \times 10^5 \end{aligned}$$

进行尾数加减运算前, 必须“对阶”! 最后还要考虑舍入
计算机内部的二进制运算也一样!

“对阶”操作: 目的是使两数阶码相等

- 小阶向大阶看齐, 阶小的那个数的尾数右移, 右移位数等于两个阶码差的绝对值
- IEEE 754尾数右移时, 要将隐含的“1”移到小数部分, 高位补0, 移出的低位保留到特定的“附加位”上

问题：如何对阶？

通过计算 $[\Delta E]_{\text{补}}$ 来判断两数的阶差：

$$[\Delta E]_{\text{补}} = [Ex - Ey]_{\text{补}} = [Ex]_{\text{移}} + [-[Ey]_{\text{移}}]_{\text{补}} \pmod{2^n}$$

问题：在 ΔE 为何值时无法根据 $[\Delta E]_{\text{补}}$ 来判断阶差？ 溢出时！

例：4位移码， $Ex=7$ ， $Ey=-7$ ，则 $[\Delta E]_{\text{补}}=1111+1111=1110$ ， $\Delta E<0$ ，错

问题：对IEEE754 SP格式来说， $|\Delta E|$ 大于多少时，结果就一定等于阶大的那个数（即小数被大数吃掉）？ 25！

1.xx...x $\rightarrow$ 0.00...0 01xx...x(右移25位后，附加位一定被丢弃)

问题：IEEE754 SP格式的偏置常数是127，这会不会影响阶码运算电路的复杂度？ 对计算 $[Ex - Ey]_{\text{补}} \pmod{2^n}$ 没有影响

$$\begin{aligned} [\Delta E]_{\text{补}} &= 256 + Ex - Ey = 256 + 127 + Ex - (127 + Ey) \\ &= 256 + [Ex]_{\text{移}} - [Ey]_{\text{移}} = [Ex]_{\text{移}} + [-[Ey]_{\text{移}}]_{\text{补}} \pmod{256} \end{aligned}$$

浮点数加减法的计算过程：

(假定： X_m 、 Y_m 分别是X和Y的尾数， X_e 和 Y_e 分别是X和Y的阶码)

- (1) 求阶差： $\Delta e = Y_e - X_e$ (若 $Y_e > X_e$ ，则结果的阶码为 Y_e)
- (2) 对阶：将 X_m 右移 Δe 位，尾数变为 $X_m * 2^{X_e - Y_e}$ (保留右移部分**附加位**)
- (3) 尾数加减： $X_m * 2^{X_e - Y_e} \pm Y_m$
- (4) 规格化：
当尾数高位为0，则需左规：尾数左移一次，阶码减1，直到MSB为1
每次阶码减1后要判断阶码是否下溢（比最小可表示的阶码还要小）
当尾数最高位有进位，需右规：尾数右移一次，阶码加1，直到MSB为1
每次阶码加1后要判断阶码是否上溢（比最大可表示的阶码还要大）
阶码溢出异常处理：阶码上溢，则结果溢出

问题：为何IEEE 754 加减运算右规时最多只需一次？

- (5) 如果尾数比规定位数长（有附加位），则需考虑舍入（有多种舍入方式）
- (6) 若运算结果尾数是0，则需要将阶码也置0。为什么？

尾数为0说明结果应该为0（阶码和尾数为全0）。

判断是否溢出：

- 1、若是阶码全为1，则上溢
- 2、若是尾数全为1，则下溢

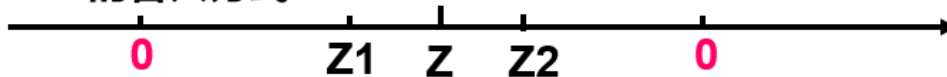
在对阶的时候，有可能小的数在右规的时候，有一部分尾数会丢失

--> 引入附加位，让这些尾数掉到附加位里面，后面计算的时候可以作为**舍入的依据**

IEEE 754 标准下的舍入方式：

- 1、就近舍入：舍入为最近可表示的数。当附加位为01，舍弃；当附加位为11，入一位；当附加位为10，让最后的结果是偶数即可
- 2、向正无穷的方向舍入
- 3、向负无穷的方向舍入
- 3、向0舍入，即直接截断尾数

IEEE 754的舍入方式



(Z1和Z2分别是结果Z的最近的可表示的左、右两个数)

(1) 就近舍入：舍入为最近可表示的数

非中间值：0舍1入；

中间值：强迫结果为偶数

例如：附加位为

01：舍

11：入

10：(强迫结果为偶数)

例：1.1101**11** → 1.1110; 1.1101**01** → 1.1101;
1.1110**10** → 1.1110; 1.1111**10** → 10.0000;
1.1101**10** → 1.1110

(2) 朝 $+\infty$ 方向舍入：舍入为Z2(正向舍入)

[BACK](#)

(3) 朝 $-\infty$ 方向舍入：舍入为Z1(负向舍入)

(4) 朝0方向舍入：截去。正数：取Z1；负数：取Z2

无符号数的乘法运算

原码的乘法

可以用加法器+移位器来实现，通过不断地加法和当前部分积的右移，就可以实现乘法运算

原码乘法运算

设 $x = 0.1101$, $y = 0.1011$ 求 $x * y$

进位	部分积	乘数	部分积初始化为 0
	0 0 0 0 0	1 0 1 1	
+X	0 1 1 0 1		乘数最低位为 1，加上被乘数
	0 1 1 0 1	1 0 1 1	部分积右移，前面补 0
	0 0 1 1 0	1 1 0 1	乘数最低位为 1，加上被乘数
+X	0 1 1 0 1		
	1 0 0 1 1	1 1 0 1	部分积右移，前面补 0
	0 1 0 0 1	1 1 1 0	乘数最低位为 0，加上 0
+0	0 0 0 0 0		
	0 1 0 0 1	1 1 1 0	部分积右移，前面补 0
	0 0 1 0 0	1 1 1 1	乘数最低位为 1，加上被乘数
+X	0 1 1 0 1		
	1 0 0 0 1	1 1 1 1	部分积右移，前面补 0
	0 1 0 0 0	1 1 1 1	运算四次结束，数值部分运算

补码的乘法

使用 Booth 算法来实现

比如： $X * Y$ ， Y 的二进制表示的最后两位为 y_1, y_0

当 $y_1 = 1, y_0 = 0$ ，对当前的结果减去被乘数，即减去 X 。然后右移一位

当 $y_1 = 1, y_0 = 1$ ，直接右移一位

当 $y_1 = 0, y_0 = 1$ ，对当前的结果加上被乘数，即加上 X 。然后右移一位

当 $y_1 = 0$, $y_0 = 0$, 直接右移一位

已知 $[X]_{补} = 1\ 101$, $[Y]_{补} = 0\ 110$, 计算 $[X \times Y]_{补}$ $[-X]_{补} = 0011$

$X = -3$, $Y = 6$, $X \times Y = -18$, $[X \times Y]_{补}$ 应等于11101110或结果溢出

P	Y	y_1	说明
0000	0110	0	设 $y_1 = 0$, $[P_0]_{补} = 0$
		$\rightarrow 1$	$y_0 y_1 = 00$, P、Y 直接右移一位
0000	0011	0	得 $[P_1]_{补}$
+0011			$y_1 y_0 = 10$, $+[-X]_{补}$
0011		$\rightarrow 1$	P、Y 同时右移一位
0001	1001	1	得 $[P_2]_{补}$
		$\rightarrow 1$	$y_2 y_1 = 11$, P、Y 直接右移一位
0000	1100	1	得 $[P_3]_{补}$
+1101			$y_3 y_2 = 01$, $+ [X]_{补}$
1101		$\rightarrow 1$	P、Y 同时右移一位
1110	1110	0	得 $[P_4]_{补}$

如何判断结果是否溢出?
高4位是否为符号位!

验证：当 $X \times Y$ 取8位时，结果 -0010010B=-18；取4位时，结果溢出 2025/7/25

乘法是否溢出的判断：

- 无符号：若高n位全0，则不溢出，否则溢出
- 带符号：若高n位全0或全1且等于低n位的最高位，则不溢出

除法运算

计算机内部无符号数除法，与手算一样，通过被除数（中间余数）减除数来得到每一位商
基本操作为减（加）法和移位，故可与乘法合用同一套硬件

开始除法之前，要先试商

对于无符号整数运算，若试商为1，则说明商有n+1位数，会溢出

若是浮点数中尾数原码小数运算，第一次试商为1，则说明尾数部分有“溢出”，可通过浮点数的“右规”消除“溢出”。

计算机中使用 恢复余数的除法 来计算：

算法要点：

- 1. 符号单独处理： $Q_s = X_s \oplus Y_s$ ，取绝对值相除。
- 2. 被除数减去除数，若不溢出（定点小数，要求商小于1）运算开始。
- 3. 若余数为正，表示够减，上商1为1，余数左移一位，减去除数；若余数为负，表示不够减，上商为0，余数加上除数（恢复余数），再左移一位减去除数；
- 4. 重复第3步，直到得到所要求位数的商为止

恢复余数除法

已知 $[X]_{原} = 0.1011$
 $[Y]_{原} = 1.1101$
用恢复余数法计算 $[X/Y]_{原}$
解：商的符号位： $0 \oplus 1 = 1$
减法操作用补码加法实现，是否够减通过中间余数的符号来判断，所以中间余数要加一位符号位。
 $[X]_{补} = 0.1011$
 $[Y]_{补} = 0.1101$
 $[-Y]_{补} = 1.0011$
小数在低位扩展0

思考：若实现无符号数相除，即1011除以1101，则有何不同？结果是什么？
被除数高位补0，1011除以1101，结果等于0

余数寄存器 R	余数/商寄存器 Q	说 明	用于判断是否溢出
01011	0000□	开始 $R_0 = X$	
+10011		$R_1 = X - Y$	
11110	00000	$R_1 < 0$ ，则 $q_4 = 0$	
+01101		恢复余数： $R_1 = R_1 + Y$	
01011		得 R_1	
10110	0000□	$2R_1$ （R 和 Q 同时左移，空出一位商）	
+10011		$R_2 = 2R_1 - Y$	
01001	00001	$R_2 > 0$ ，则 $q_3 = 1$	
10010	0001□	$2R_2$ （R 和 Q 同时左移，空出一位商）	
+10011		$R_3 = 2R_2 - Y$	
00101	00011	$R_3 > 0$ ，则 $q_2 = 1$	
01010	0011□	$2R_3$ （R 和 Q 同时左移，空出一位商）	
+10011		$R_4 = 2R_3 - Y$	
11101	00110	$R_4 < 0$ ，则 $q_1 = 0$	
+01101		恢复余数： $R_4 = R_4 + Y$	
01010	00110	得 R_4	
10100	0110□	$2R_4$ （R 和 Q 同时左移，空出一位商）	
+10011		$R_5 = 2R_4 - Y$	
00111	01101	$R_5 > 0$ ，则 $q_0 = 1$	

商的最高位为0，说明没有溢出，商的数值部分为：1101。
所以， $[X/Y]_{原} = 1.1101$ （最高位为符号位），余数为 0.0111×2^4 。

计算机中使用 不恢复余数的除法 来计算：

原码不恢复余数法算法

- 1. 符号单独处理： $Q_s = X_s \oplus Y_s$ ，取绝对值相除；
- 2. 被除数减去除数；
- 3. 若所得余数为正，相应位上商为1，余数左移一位，减去除数；若余数为负，相应位上商为0，余数左移一位，加上除数；
- 4. 重复第3步，直到求得所要求的商的各位为止

注意：最后一次上商为“0”的话，需要“纠余”处理，即把试商时被减掉的除数加回去，恢复真正的余数。

已知 $[X]_{原} = 0.1011$
 $[Y]_{原} = 1.1101$
用加减交替法计算 $[X/Y]_{原}$
解： $[|X|]_{补} = 0.1011$
 $[|Y|]_{补} = 0.1101$
 $[-|Y|]_{补} = 1.0011$

“加减交替法”的要点：
负、0、加
正、1、减

得到的结果与恢复余数法一样！

余数寄存器 R	余数/商寄存器 Q	说 明
01011	0000□	开始 $R_0 = X$
+10011		$R_1 = X - Y$
11110	00000	$R_1 < 0$ ，则 $q_4 = 0$ ，没有溢出
11100	0000□	$2R_1$ (R 和 Q 同时左移，空出一位商)
+01101		$R_2 = 2R_1 + Y$
01001	00001	$R_2 > 0$ ，则 $q_3 = 1$
10010	0001□	$2R_2$ (R 和 Q 同时左移，空出一位商)
+10011		$R_3 = 2R_2 - Y$
00101	00011	$R_3 > 0$ ，则 $q_2 = 1$
01010	0011□	$2R_3$ (R 和 Q 同时左移，空出一位商)
+10011		$R_4 = 2R_3 - Y$
11101	00110	$R_4 < 0$ ，则 $q_1 = 0$
11010	0110□	$2R_4$ (R 和 Q 同时左移，空出一位商)
+01101		$R_5 = 2R_4 + Y$
00111	01101	$R_5 > 0$ ，则 $q_0 = 1$

注意：余数的符号始终与被除数的符号一致！！